

PROJETO 3: MINIX

Izaque Vieira de Souza¹, Olávio Gonçalves de Almeida¹, Wisley Kilder da Silva Nogueira¹

¹Departamento de Ciência da Computação – Universidade Federal de Goiás (UFG)

izaquevieira@outlook.com.br, olaviog.almeida@gmail.com,
w.kilder@hotmail.com

1. Tarefa 1 – Instalação do sistema

1.1. Descrição

A primeira tarefa consiste na instalação do SO Minix. Para esta atividade foi usada a versão 3.1.8 do Minix, com as configurações de máquinas exigidas (128 MB de memória RAM e 2 GB de armazenamento). O software utilizado para a tarefa de virtualização do SO foi o VirtualBox, e a seguir será apresentado como foi realizada a instalação do SO, bem como os softwares necessários.

1.2. Instalação do SO

O iniciar a máquina, é necessário fazer o login com o usuário root, e logo em seguida executa o comando *setup* para que a instalação possa prosseguir. O processo está descrito na Imagem 1. A Figura 1 apresenta como foi configurada a rede da máquina virtual, e logo em seguida na Figura 2, é mostrada a instalação do SO.

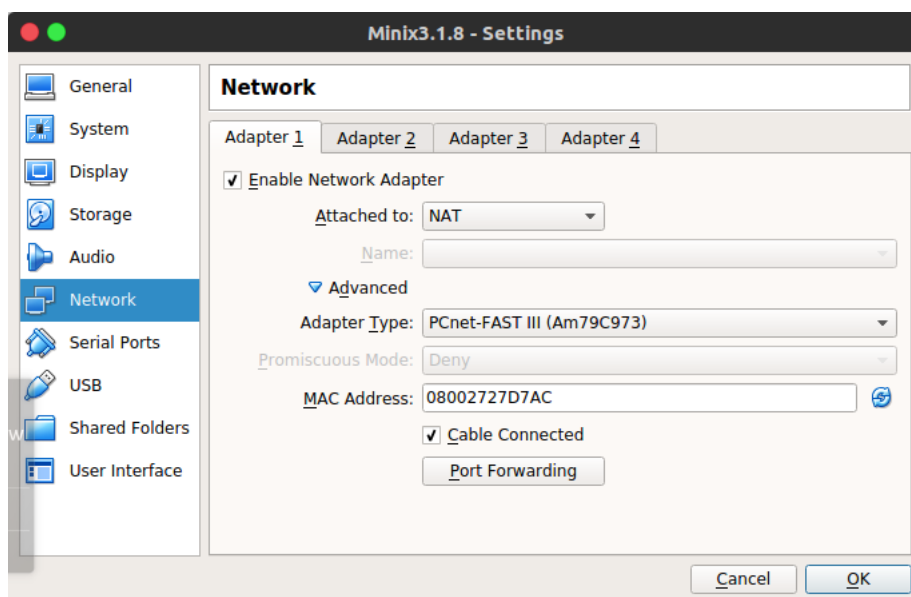


Figura 1 – Configuração da rede da VM

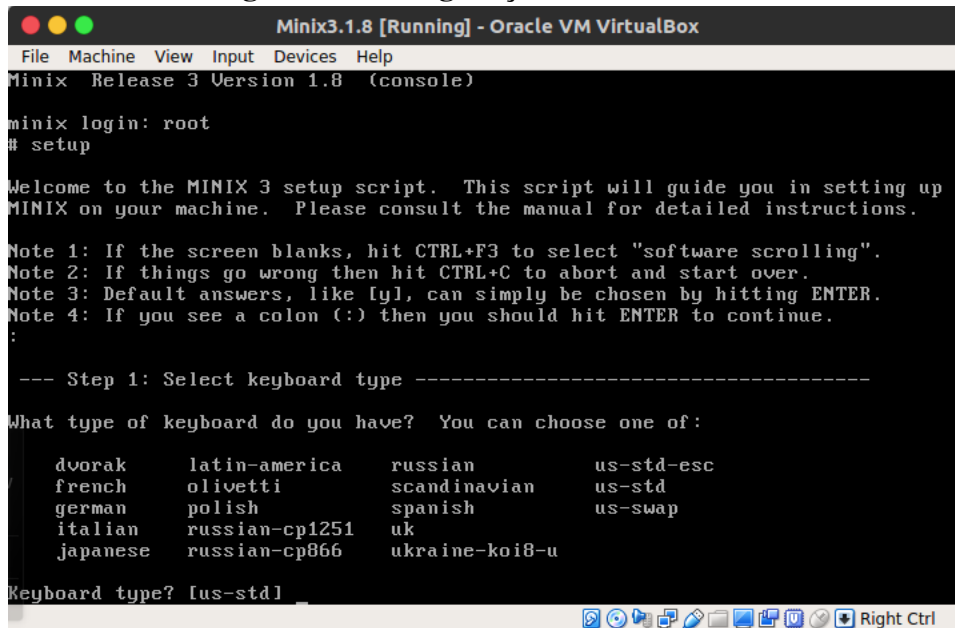


Figura 2 – Instalação do SO

Após a instalação ser concluída, deve-se desligar a máquina e iniciar ela no disco que foi instalada, através do comando *boot b0p0*.

1.3. Instalação dos *softwares* necessários

Seguindo com a preparação do ambiente, será descrito abaixo como os *softwares* foram instalados no SO.

Primeiramente, foi definido uma senha para o usuário *root*, através do comando *passwd*. Em seguida, foi instalado os seguintes *softwares*: *ssh*, *bash-4.1nbl* e *nano*, utilizando os seguintes comandos, respectivamente: *pkgin install openssh*, *pkgin install bash-4.1nbl*, *pkgin install nano*. Na Figura 3 é possível perceber pela resposta “*google.com is alive*” que a rede está funcionando. Em seguida, a Figura 4 mostra que o *ssh* foi instalado e funciona, e a Figura 5 apresenta o *bash* instalado e funcionando. A Figura 6 apresenta a conexão *ssh* estabelecida do sistema hospedeiro com a máquina virtual através do comando *ssh root@localhost -p 2020*, e na Figura 7 mostra que transmissão de um arquivo *teste.txt* da máquina hospedeira para a máquina virtual foi feita corretamente, através do *ssh*, utilizando o comando *scp -P2020 teste.txt root@localhost:/root*. Por fim, a Figura 8 apresenta a criação de um usuário “*olavio*”, assim como a definição de uma senha e configurando o *bash* como *shell* padrão do usuário. A criação do usuário, a definição do *bash* como *shell* padrão foram feitos pelos seguintes comandos, respectivamente: *adduser olavio other /usr/olavio*; *chsh olavio /usr/pkg/bin/bash*.

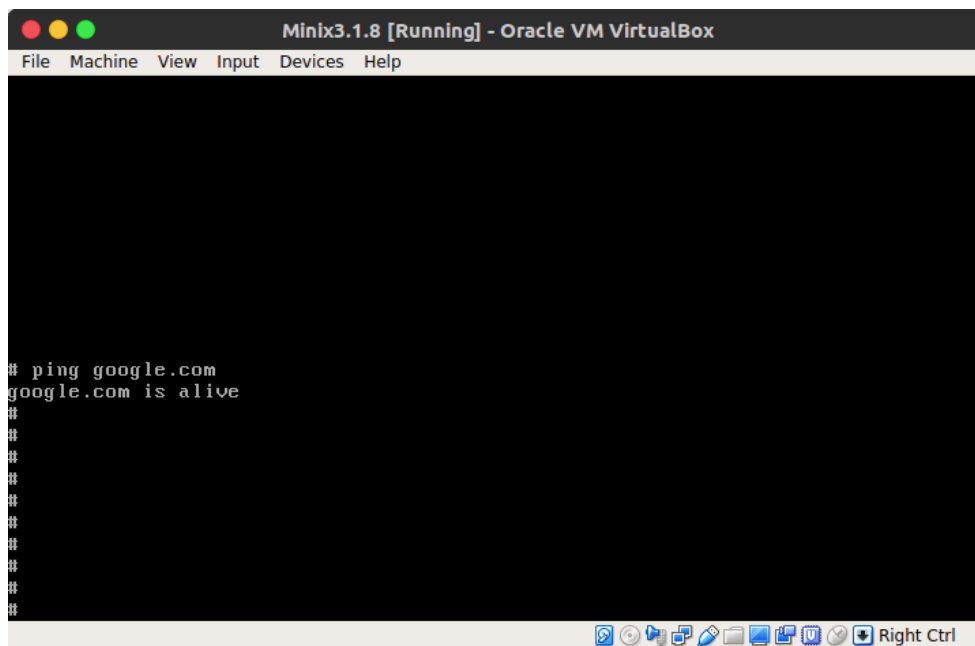


Figura 3 – demonstração da rede funcionando

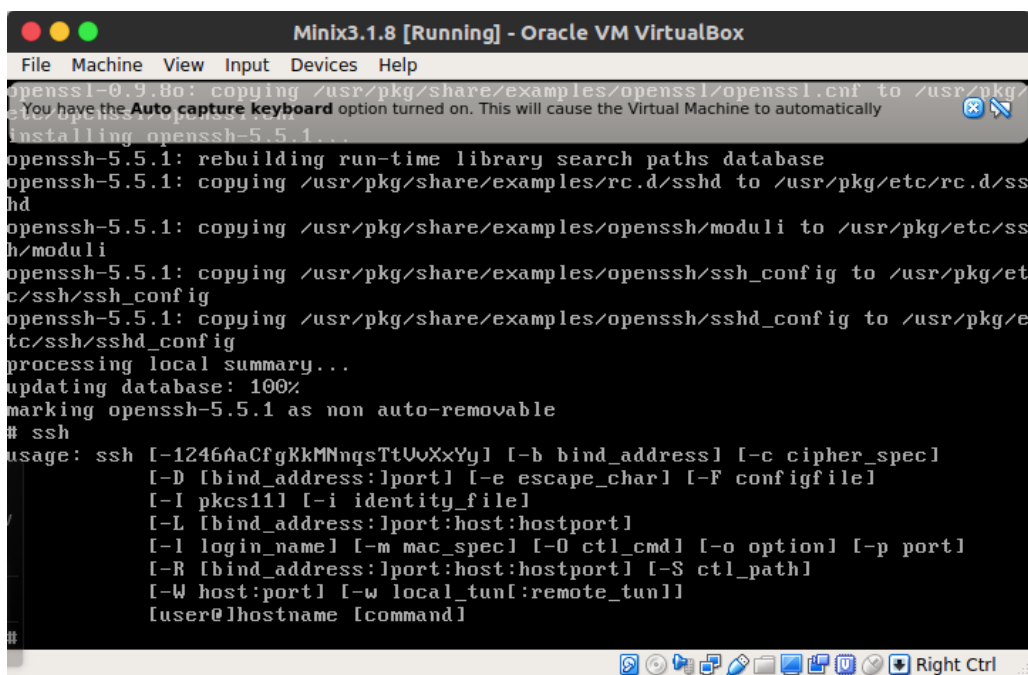
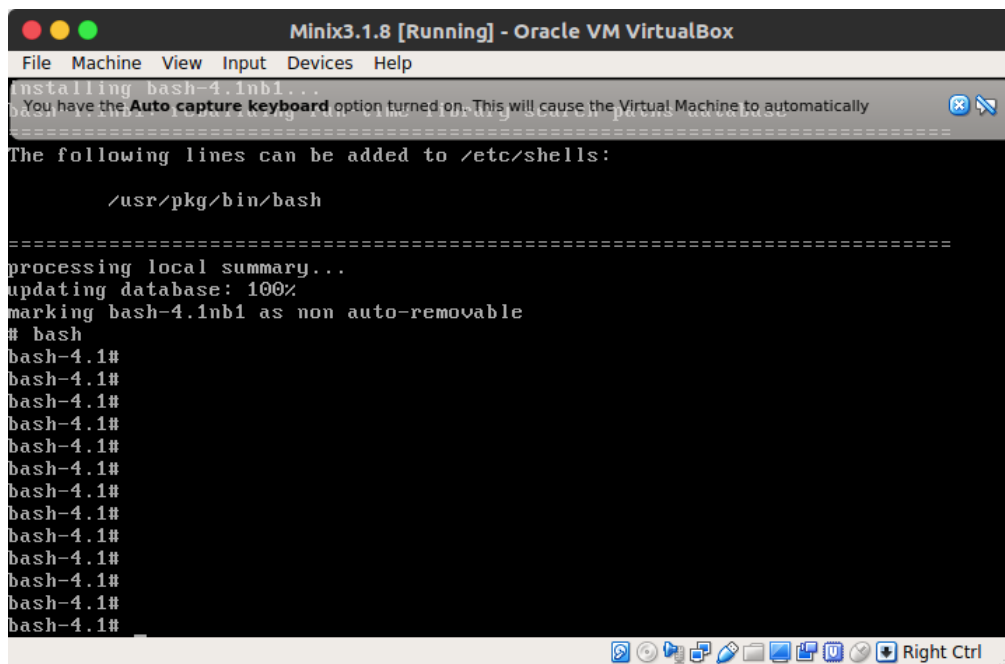


Figura 4 – ssh funcionando corretamente

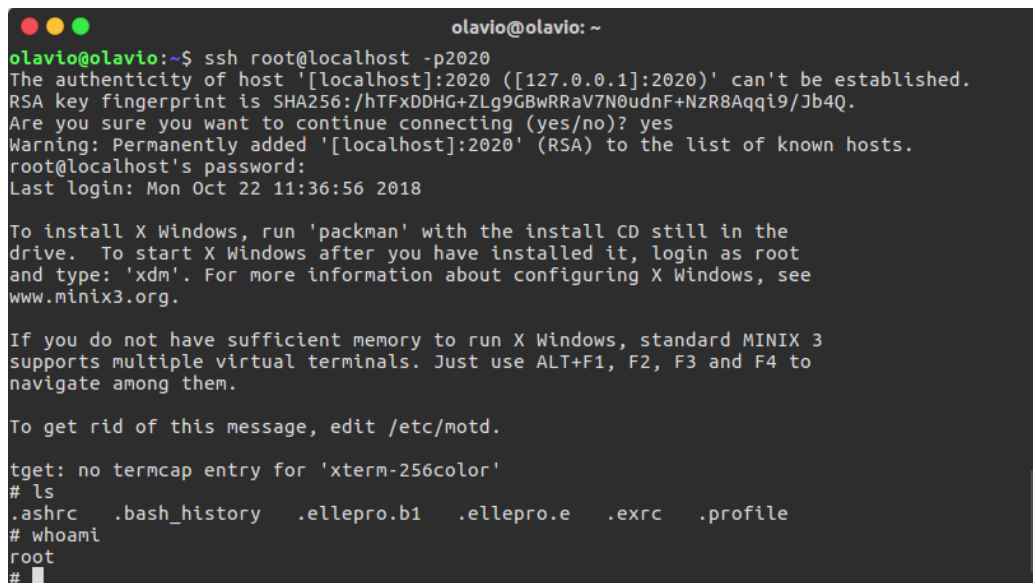


```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
installing bash-4.1nb1...
You have the Auto capture keyboard option turned on. This will cause the Virtual Machine to automatically
capture keyboard input. Pressing Ctrl+Alt+P will toggle this option.
=====
The following lines can be added to /etc/shells:

    /usr/pkg/bin/bash

=====
processing local summary...
updating database: 100%
marking bash-4.1nb1 as non auto-removable
# bash
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
bash-4.1#
Right Ctrl
```

Figura 5 – *bash* funcionando corretamente



```
olavio@olavio: ~
olavio@olavio:~$ ssh root@localhost -p2020
The authenticity of host '[localhost]:2020 ([127.0.0.1]:2020)' can't be established.
RSA key fingerprint is SHA256:/hTFxDDHG+ZLg9GBwRRaV7N0udnF+NzR8Aqqi9/Jb4Q.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[localhost]:2020' (RSA) to the list of known hosts.
root@localhost's password:
Last login: Mon Oct 22 11:36:56 2018

To install X Windows, run 'packman' with the install CD still in the
drive. To start X Windows after you have installed it, login as root
and type: 'xdm'. For more information about configuring X Windows, see
www.minix3.org.

If you do not have sufficient memory to run X Windows, standard MINIX 3
supports multiple virtual terminals. Just use ALT+F1, F2, F3 and F4 to
navigate among them.

To get rid of this message, edit /etc/motd.

tget: no termcap entry for 'xterm-256color'
# ls
.lashrc  .bash_history  .ellepro.b1  .ellepro.e  .exrc  .profile
# whoami
root
#
```

Figura 6 – conexão *ssh* do hospedeiro com a VM

```
olavio@olavio: ~
supports multiple virtual terminals. Just use ALT+F1, F2, F3 and F4 to
navigate among them.

To get rid of this message, edit /etc/motd.

tget: no termcap entry for 'xterm-256color'
# ls
.ashrc .bash_history .ellepro.b1 .ellepro.e .exrc .profile
# whoami
root
#
# exit
Connection to localhost closed.
olavio@olavio:~$ scp -P2020 teste.txt root@localhost:/root
root@localhost's password:
teste.txt                               100% 17      0.0KB/s   00:00
olavio@olavio:~$
olavio@olavio:~$
olavio@olavio:~$
olavio@olavio:~$
olavio@olavio:~$
olavio@olavio:~$
olavio@olavio:~$
olavio@olavio:~$
```

Figura 7 – transmissão de um arquivo do hospedeiro para a VM

```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
# chfn olavio Olavio Almeida
chfn [user] fullname
# chfn olavio Olavio
# chsh olavio /usr/pkg/bin/bash
# exit

Minix Release 3 Version 1.8 (console)
10.0.2.15 login: olavio

To install X Windows, run 'packman' with the install CD still in the
drive. To start X Windows after you have installed it, login as root
and type: 'xdm'. For more information about configuring X Windows, see
www.minix3.org.

If you do not have sufficient memory to run X Windows, standard MINIX 3
supports multiple virtual terminals. Just use ALT+F1, F2, F3 and F4 to
navigate among them.

To get rid of this message, edit /etc/motd.

-bash: [: too many arguments
-bash-4.1$ whoami
olavio
-bash-4.1$
```

Figura 8 – usuário criado

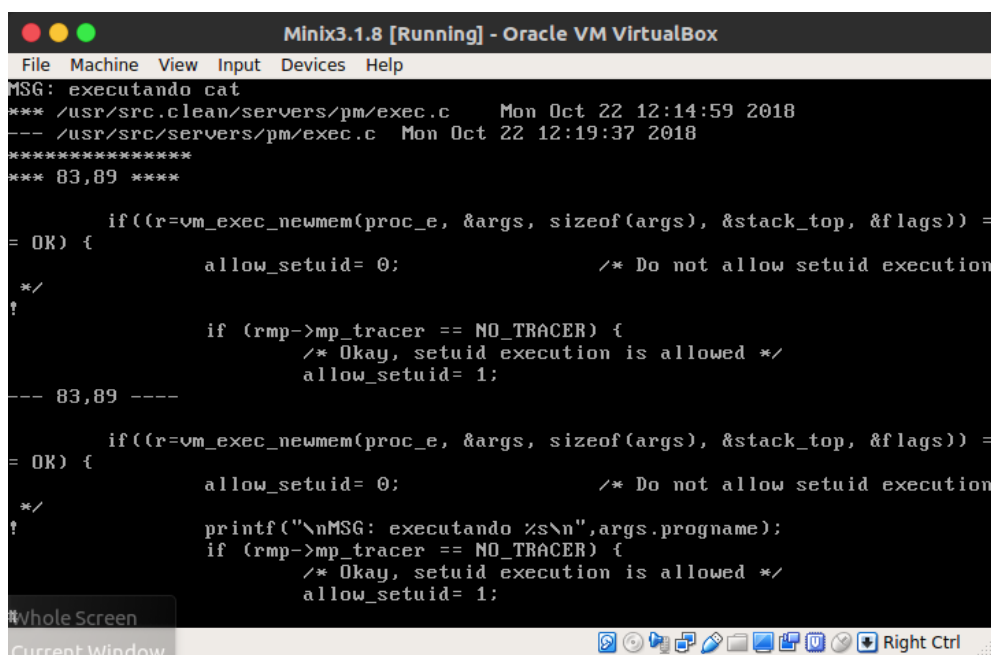
2. Tarefa 2 – Recompilando o kernel no Minix

2.1. Descrição da tarefa

O primeiro passo da tarefa consiste em recompilar o *kernel* do Minix, que pode ser feito após acessar o diretório `/usr/src/tools` e executar o comando `make hdbboot`. Feito

isso, deve-se reiniciar o sistema, e ao entrar, deve-se selecionar a opção “*Start custom kernel*”. A segunda parte desta tarefa consiste em modificar o kernel de modo que seja mostrado na tela qual tarefa está sendo executada.

Após recompilar o *kernel* do Minix o mesmo foi reiniciado e iniciado no modo *custom*. Para prosseguir, foi alterado no código *exec.c* presente no diretório */usr/src/servers/pm*. O comando *diff -c /usr/src/servers/pm/exec.c /usr/src.clean/servers/pm/exec.c > diferenca.patch* foi utilizado para gerar um arquivo contendo a diferença entre o arquivo original e o arquivo alterado. O conteúdo do arquivo *diferenca.patch* é apresentado na Figura 9 e o resultado após recompilar o *kernel* novamente e executar o SO é apresentado na Figura 10.



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
MSG: executando cat
*** /usr/src.clean/servers/pm/exec.c  Mon Oct 22 12:14:59 2018
--- /usr/src/servers/pm/exec.c      Mon Oct 22 12:19:37 2018
*****
*** 83,89 ****
+
+     if ((r=vm_exec_newmem(proc_e, &args, sizeof(args), &stack_top, &flags)) =
= OK) {
+         allow_setuid= 0;                /* Do not allow setuid execution
+
+     */
+     ?
+         if (rmp->mp_tracer == NO_TRACER) {
+             /* Okay, setuid execution is allowed */
+             allow_setuid= 1;
+
+ --- 83,89 ----
+
+     if ((r=vm_exec_newmem(proc_e, &args, sizeof(args), &stack_top, &flags)) =
= OK) {
+         allow_setuid= 0;                /* Do not allow setuid execution
+
+     */
+     ?
+         printf("\nMSG: executando %s\n",args.progname);
+         if (rmp->mp_tracer == NO_TRACER) {
+             /* Okay, setuid execution is allowed */
+             allow_setuid= 1;
```

Figura 9 – Conteúdo do arquivo *diferenca.patch*

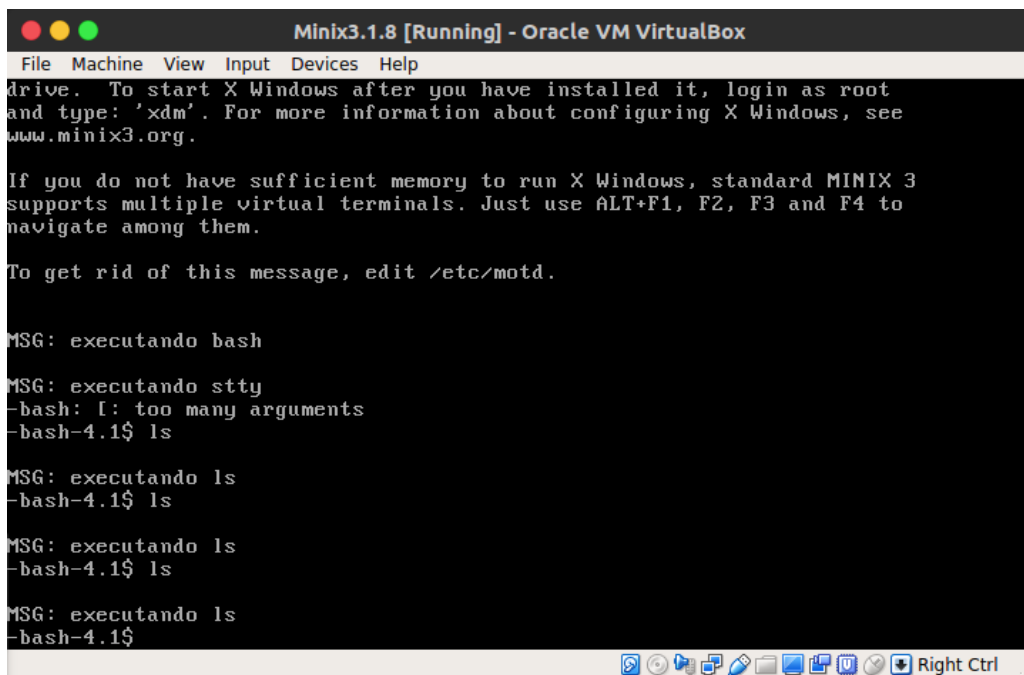
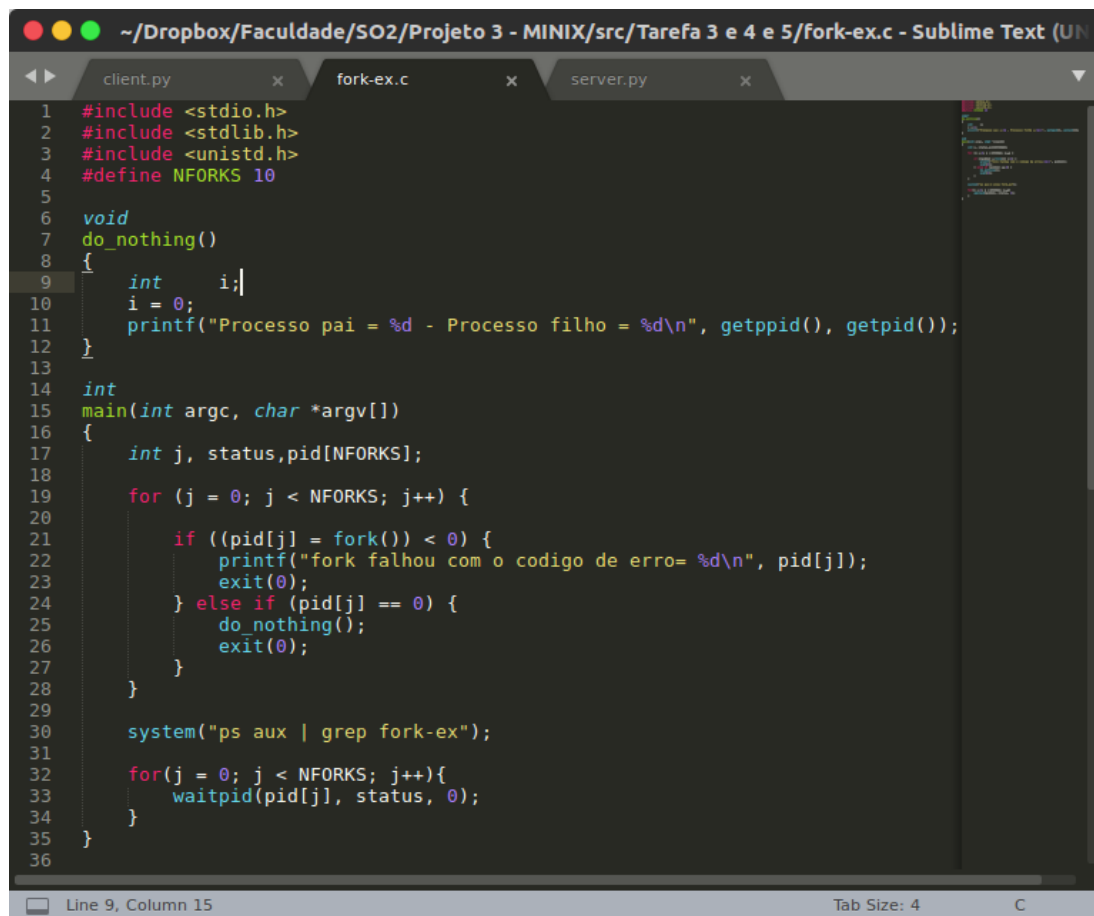


Figura 10 – resultado após recompilar o *kernel*

3. Gerenciamento de processos

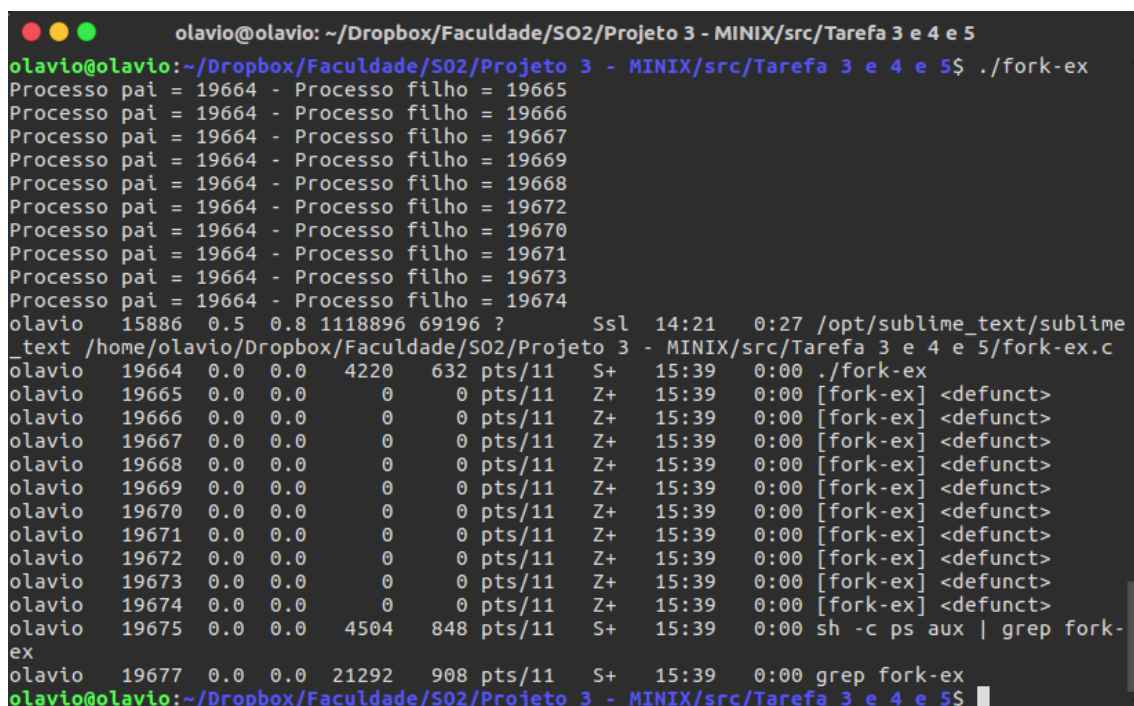
Respondendo as questões teóricas:

- a) A chamada de sistema *fork()* é usada para criar processos filhos a partir de um processo pai. Neste caso, o processo pai é o arquivo *fork-ex.c*, pois ele faz a chamada do *fork()* para criar os processos filhos. No arquivo *fork-ex.c*, existe uma linha onde *pid = fork()*, e logo após é feita uma verificação se o valor de *pid* é negativo. Caso ele seja negativo, indica que a chamada *fork()* não foi feita com sucesso, apresentando uma mensagem de erro.
- b) O código foi modificado como pedido, para que a função *do_nothing()* mostrasse na tela o número do processo pai e dos processos filhos criados. Foi feito também uma alteração para que o fosse mostrado na tela através do comando *ps* os processos criados. O arquivo após as alterações feitas é apresentado na Figura 11. A Figura 12 apresenta o resultado obtido ao executar o arquivo *fork-ex.c* após as modificações.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #define NFORKS 10
5
6 void
7 do_nothing()
8 {
9     int i;
10    i = 0;
11    printf("Processo pai = %d - Processo filho = %d\n", getppid(), getpid());
12 }
13
14 int
15 main(int argc, char *argv[])
16 {
17     int j, status, pid[NFORKS];
18
19     for (j = 0; j < NFORKS; j++) {
20
21         if ((pid[j] = fork()) < 0) {
22             printf("fork falhou com o codigo de erro= %d\n", pid[j]);
23             exit(0);
24         } else if (pid[j] == 0) {
25             do_nothing();
26             exit(0);
27         }
28     }
29
30     system("ps aux | grep fork-ex");
31
32     for(j = 0; j < NFORKS; j++){
33         waitpid(pid[j], status, 0);
34     }
35 }
36
```

Figura 11 – arquivo *fork-ex.c*



```
olavio@olavio: ~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$ ./fork-ex
Processo pai = 19664 - Processo filho = 19665
Processo pai = 19664 - Processo filho = 19666
Processo pai = 19664 - Processo filho = 19667
Processo pai = 19664 - Processo filho = 19669
Processo pai = 19664 - Processo filho = 19668
Processo pai = 19664 - Processo filho = 19672
Processo pai = 19664 - Processo filho = 19670
Processo pai = 19664 - Processo filho = 19671
Processo pai = 19664 - Processo filho = 19673
Processo pai = 19664 - Processo filho = 19674
olavio 15886 0.5 0.8 1118896 69196 ? Ssl 14:21 0:27 /opt/sublime_text/sublime
_text /home/olavio/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5/fork-ex.c
olavio 19664 0.0 0.0 4220 632 pts/11 S+ 15:39 0:00 ./fork-ex
olavio 19665 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19666 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19667 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19668 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19669 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19670 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19671 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19672 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19673 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19674 0.0 0.0 0 0 pts/11 Z+ 15:39 0:00 [fork-ex] <defunct>
olavio 19675 0.0 0.0 4504 848 pts/11 S+ 15:39 0:00 sh -c ps aux | grep fork-
ex
olavio 19677 0.0 0.0 21292 908 pts/11 S+ 15:39 0:00 grep fork-ex
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$
```

Figura 12 – execução do arquivo *fork-ex.c*

- c) Na implementação feita, foi criada uma constante que é usada para generalizar a quantidade de escritores do problema. A constante foi chamada *QTD_WRITERS*. Na Figura 13 é possível ver o resultado e logo em seguida o código feito (Figura 14).

```
olavio@olavio: ~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5
Escrutor 22461 - Mensagem:
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$ ./
pipe

Escrutor 22464 - Mensagem:
escrevendo.....
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$

Escrutor 22465 - Mensagem:
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$ gc
c pipe.c -o pipe
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$ ./
pipe
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$

Escrutor 22484 - Mensagem:
escrevendo.....

Escrutor 22485 - Mensagem:
escrevendo.....

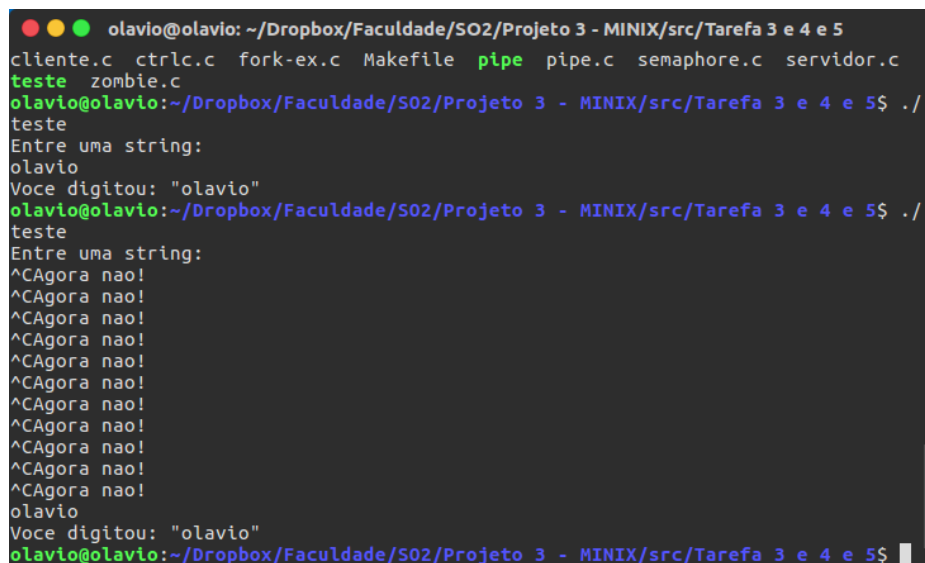
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$
```

Figura 13 – resultado questão 3-c

```
~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5/pipe.c - Subl
client.py x pipe.c x server.py x
3 #include <string.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 #define QTD_WRITERS 2
8 #define BUFFER 256
9
10 typedef struct{
11     int id, lastone;
12     char data[BUFFER];
13 }PACKET;
14
15 int main(){
16     pid_t pid;
17     int fd[2], i;
18     if(pipe(fd) < 0){
19         perror("pipe");
20         return -1;
21     }
22     for(i = 0; i < QTD_WRITERS; i++){
23         if((pid = fork()) < 0){
24             perror("fork");
25             exit(1);
26         }
27         if(pid > 0){
28             close(fd[0]);
29             PACKET escritor;
30             escritor.id = getpid();
31             strcpy(escritor.data, "escrevendo.....");
32             if(i == QTD_WRITERS-1)
33                 escritor.lastone = 1;
34             else
35                 escritor.lastone = 0;
36             write(fd[1], &escritor, sizeof(escritor));
37             exit(0);
38         }
39     }
40     PACKET leitor;
41     close(fd[1]);
42     do{
43         read(fd[0], &leitor, sizeof(leitor));
44         printf("\n\nEscrutor %d - Mensagem:\n%s\n", leitor.id, leitor.data);
45         sleep(3);
46     }while(!leitor.lastone);
47     exit(0);
48 }
```

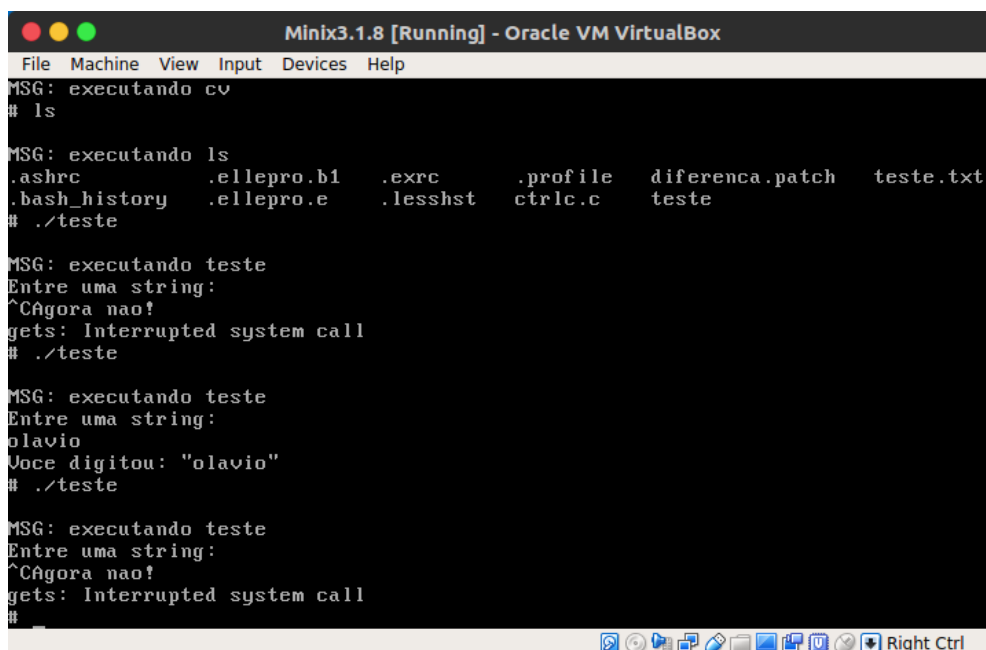
Figura 14 – código questão 3-c

- d)
- e)
- f) O código foi executado nos dois ambientes, e foi possível perceber que no Linux, o comando *Ctrl+c* não parava a execução do mesmo, já no Minix a execução do programa era interrompida ao utilizar o mesmo comando. Na Figura 15 e Figura 16 é possível ver o resultado de execução no Linux e no Minix.



```
olavio@olavio: ~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5
cliente.c ctrlc.c fork-ex.c Makefile pipe pipe.c semaphore.c servidor.c
teste zombie.c
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$ ./
teste
Entre uma string:
olavio
Voce digitou: "olavio"
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$ ./
teste
Entre uma string:
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
^CAgora nao!
olavio
Voce digitou: "olavio"
olavio@olavio:~/Dropbox/Faculdade/SO2/Projeto 3 - MINIX/src/Tarefa 3 e 4 e 5$
```

Figura 15 – execução do código *ctrlc.c* no Linux



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
MSG: executando cv
# ls
MSG: executando ls
.ashrc .ellepro.b1 .exrc .profile diferenca.patch teste.txt
.bash_history .ellepro.e .lessht ctrlc.c teste
# ./teste
MSG: executando teste
Entre uma string:
^CAgora nao!
gets: Interrupted system call
# ./teste
MSG: executando teste
Entre uma string:
olavio
Voce digitou: "olavio"
# ./teste
MSG: executando teste
Entre uma string:
^CAgora nao!
gets: Interrupted system call
#
```

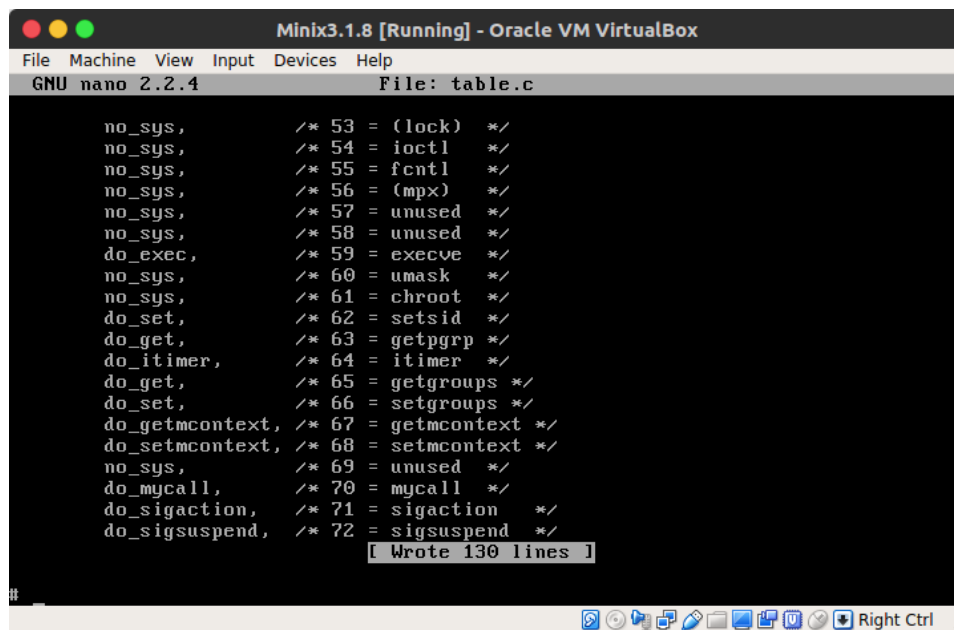
Figura 16 – execução do código *ctrlc.c* no Minix

4. Tarefa 4 – Gerenciamento de processos: processos orfãos

5. Tarefa 5 – Gerenciamento de processos: processos zumbis

6. Tarefa 6 - Criando uma chamada de sistema

- a) Os arquivos *table.c* e *proto.h* foram alterados como pedido no exercício. O arquivo *table.c* foi alterado na entrada de número 70, atribuindo a chamada de sistema a ser criada (Figura 17). O arquivo *proto.h* foi alterado para que a a protótipo da chamada de sistema fosse incluída (Figura 18).

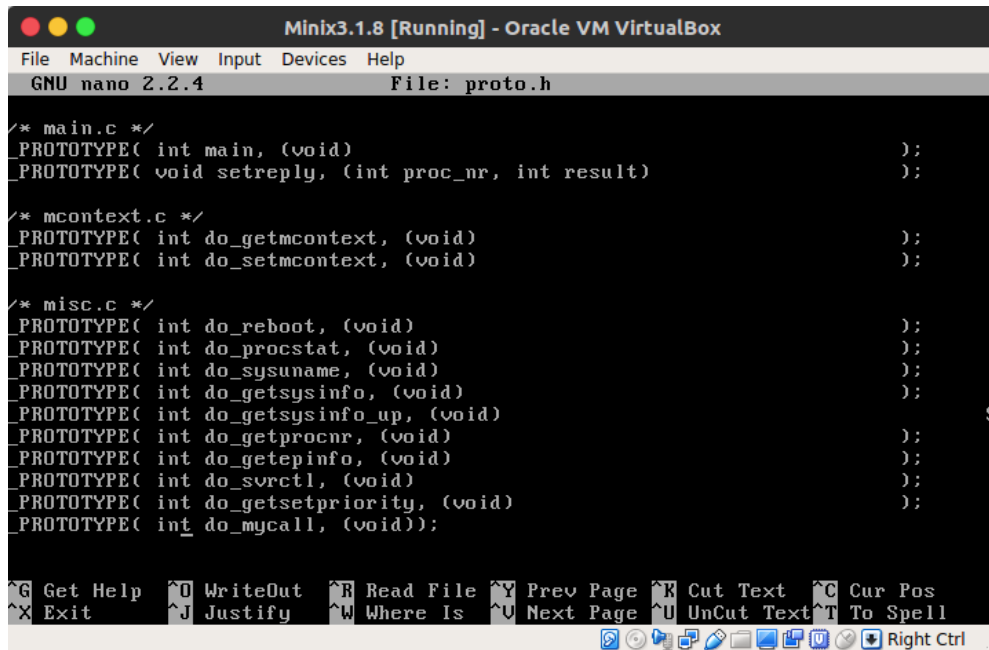


```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: table.c

no_sys,      /* 53 = (lock) */
no_sys,      /* 54 = ioctl */
no_sys,      /* 55 = fcntl */
no_sys,      /* 56 = (mpx) */
no_sys,      /* 57 = unused */
no_sys,      /* 58 = unused */
do_exec,     /* 59 = execve */
no_sys,      /* 60 = umask */
no_sys,      /* 61 = chroot */
do_set,      /* 62 = setsid */
do_get,      /* 63 = getpgrp */
do_itimer,   /* 64 = itimer */
do_get,      /* 65 = getgroups */
do_set,      /* 66 = setgroups */
do_getmcontext, /* 67 = getmcontext */
do_setmcontext, /* 68 = setmcontext */
no_sys,      /* 69 = unused */
do_mycall,   /* 70 = mycall */
do_sigaction, /* 71 = sigaction */
do_sigsuspend, /* 72 = sigsuspend */

[Wrote 130 lines]
```

Figura 17 – arquivo *table.c*



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: proto.h

/* main.c */
PROTOTYPE( int main, (void) );
PROTOTYPE( void setreply, (int proc_nr, int result) );

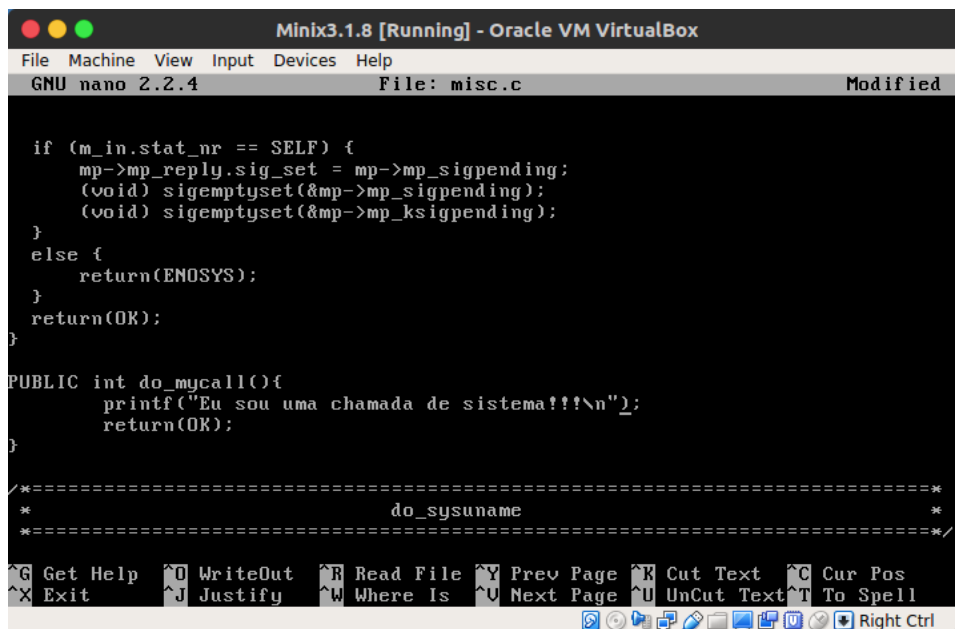
/* mcontext.c */
PROTOTYPE( int do_getmcontext, (void) );
PROTOTYPE( int do_setmcontext, (void) );

/* misc.c */
PROTOTYPE( int do_reboot, (void) );
PROTOTYPE( int do_procstat, (void) );
PROTOTYPE( int do_sysuname, (void) );
PROTOTYPE( int do_getsysinfo, (void) );
PROTOTYPE( int do_getsysinfo_up, (void) );
PROTOTYPE( int do_getprocnr, (void) );
PROTOTYPE( int do_getepinfo, (void) );
PROTOTYPE( int do_svrctl, (void) );
PROTOTYPE( int do_getsetpriority, (void) );
PROTOTYPE( int do_mycall, (void));

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
Right Ctrl
```

Figura 18 – arquivo *proto.h*

- b) A chamada de sistema *do_mycall()* foi implementada de acordo com a Figura 19, apresentada a seguir:



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: misc.c Modified

if (m_in.stat_nr == SELF) {
    mp->mp_reply.sig_set = mp->mp_sigpending;
    (void) sigemptyset(&mp->mp_sigpending);
    (void) sigemptyset(&mp->mp_ksigpending);
}
else {
    return(ENOSYS);
}
return(OK);
}

PUBLIC int do_mycall(){
    printf("Eu sou uma chamada de sistema!!!\n");
    return(OK);
}

/*=====
 * do_sysuname
 *=====*/
```

Figura 19 – implementação de *do_mycall()*

- c) O *kernel* foi recompilado, assim como as bibliotecas e os includes, e não foi gerado nenhum erro (Figura 20).

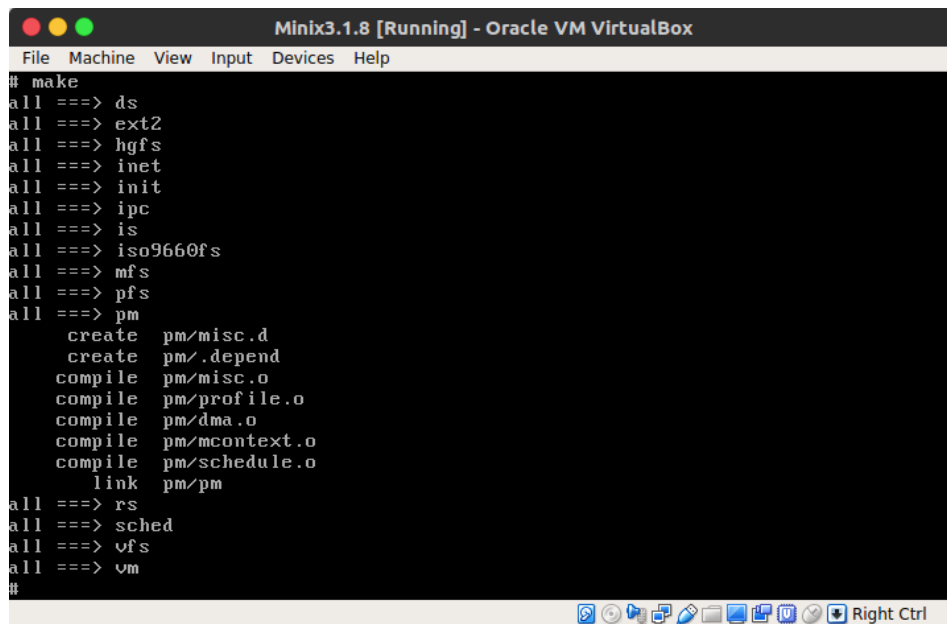


Figura 20 – saída ao recompilar o *kernel* e as bibliotecas

- d) O arquivo *callnr.h* foi alterado, para inserir a nova chamada de sistema. A alteração é apresentada em seguida na Figura 21.

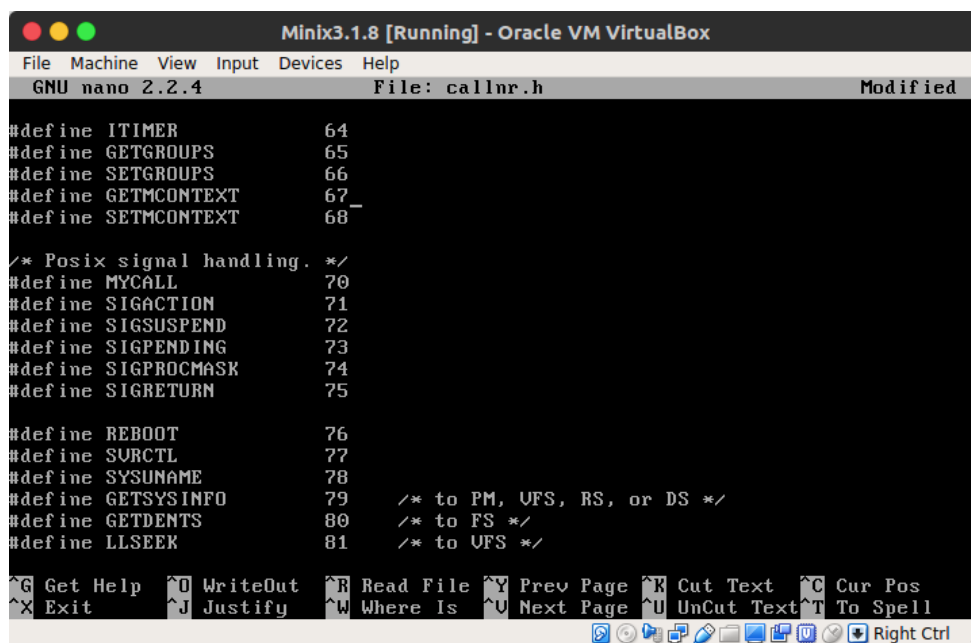
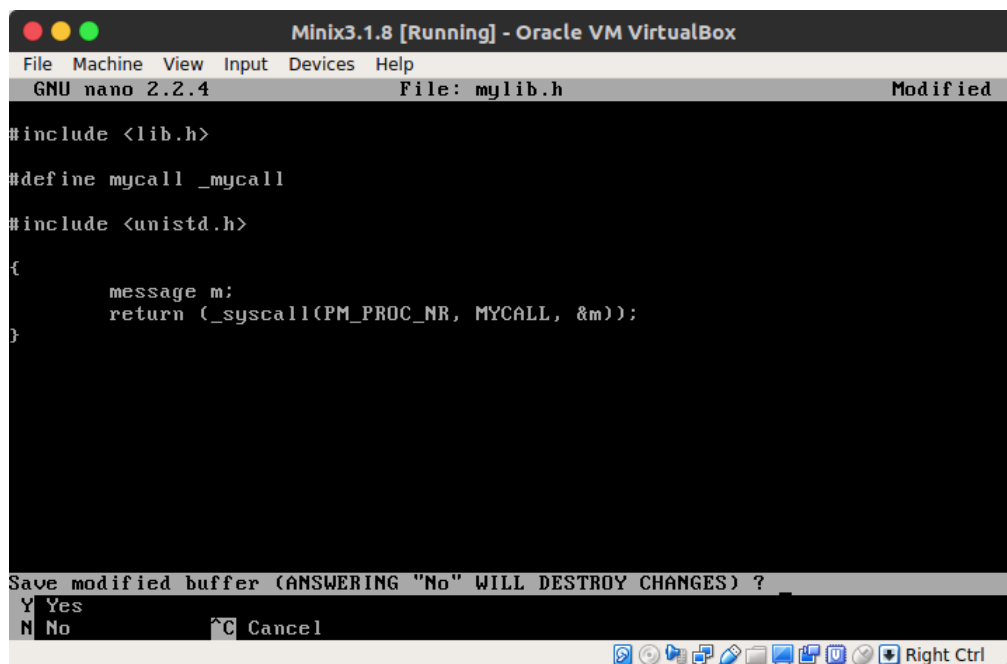


Figura 21 – arquivo *callnr.h* após alteração

- e) A nova biblioteca foi criada, e a imagem a seguir (Figura 22) apresenta o código da mesma, após algumas alterações necessárias para que funcionasse corretamente:



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: mylib.h Modified

#include <lib.h>
#define mycall _mycall
#include <unistd.h>
{
    message m;
    return (_syscall(PM_PROC_NR, MYCALL, &m));
}

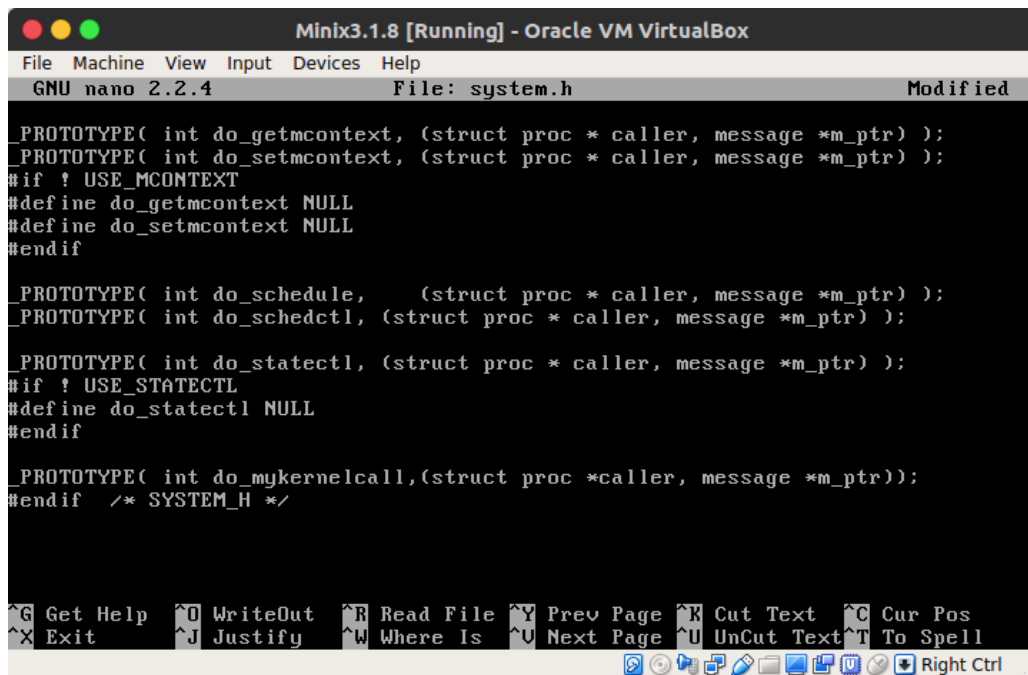
Save modified buffer (ANSWERING "No" WILL DESTROY CHANGES) ?
Y Yes
N No ^C Cancel
```

Figura 22 – código da biblioteca *mylib.h* após correções

- f) O Minix usa o message para passar parâmetros para uma chamada de sistema. Quando é feita uma *sys_call()*, ela é transformada em uma mensagem com o tipo *SYS_CALL*, que posteriormente é manipulada em uma função *do_call()*. Seguindo pelo diretório */usr/src/kernel* é possível encontrar o arquivo *system.c*, onde são criadas as mensagens *SYS_CALL*, redirecionando a mensagem para a função *do_call()*.
- g) O *kernel* foi recompilado e o sistema foi iniciado novamente, sem nenhum erro.
- h) O programa foi criado de acordo com a Figura 23, e o resultado é apresentado na Figura 24.

Figura 23 – código escrito para testar a chamada de sistema

a)



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: system.h Modified

_PROTOTYPE( int do_getmcontext, (struct proc * caller, message *m_ptr) );
_PROTOTYPE( int do_setmcontext, (struct proc * caller, message *m_ptr) );
#if ! USE_MCONTEXT
#define do_getmcontext NULL
#define do_setmcontext NULL
#endif

_PROTOTYPE( int do_schedule, (struct proc * caller, message *m_ptr) );
_PROTOTYPE( int do_schedctl, (struct proc * caller, message *m_ptr) );

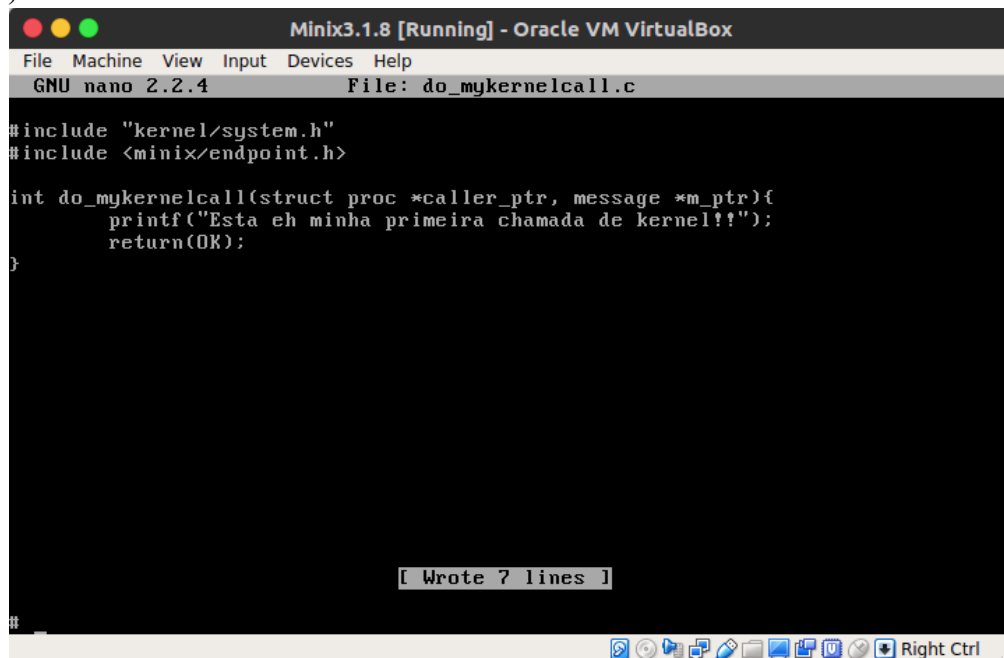
_PROTOTYPE( int do_statectl, (struct proc * caller, message *m_ptr) );
#if ! USE_STATECTL
#define do_statectl NULL
#endif

_PROTOTYPE( int do_mykernelcall, (struct proc * caller, message *m_ptr));
#endif /* SYSTEM_H */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
[Icons] Right Ctrl
```

Figura 25 – adicionando um protótipo da função

b)



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: do_mykernelcall.c

#include "kernel/system.h"
#include <minix/endpoint.h>

int do_mykernelcall(struct proc *caller_ptr, message *m_ptr){
    printf("Esta eh minha primeira chamada de kernel!!");
    return(OK);
}

[ Wrote 7 lines ]

#
```

Figura 26 – implementação da função `do_mykernelcall()`;

c)

```

Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: Makefile.inc Modified

do_sigreturn.c \
do_abort.c \
do_getinfo.c \
do_sprofile.c \
do_cprofile.c \
do_profbuf.c \
do_vmctl.c \
do_mcontext.c \
do_schedule.c \
do_schedctl.c \
do_statectl.c \
do_mykernelcall.c

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
Right Ctrl

```

Figura 27 – adicionando ao makefile

d)

```

Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: system.c Modified

/* Process management. */
map(SYS_FORK, do_fork); /* a process forked a new process */
map(SYS_EXEC, do_exec); /* update process after execute */
map(SYS_CLEAR, do_clear); /* clean up after process exit */
map(SYS_EXIT, do_exit); /* a system process wants to exit */
map(SYS_PRIUCTL, do_privctl); /* system privileges control */
map(SYS_TRACE, do_trace); /* request a trace operation */
map(SYS_SETGRANT, do_setgrant); /* get/set own parameters */
map(SYS_RUNCTL, do_runctl); /* set/clear stop flag of a process */
map(SYS_UPDATE, do_update); /* update a process into another */
map(SYS_STATECTL, do_statectl); /* let a process control its state */
map(SYS_MYKERNELCALL, do_mykernelcall);

/* Signal handling. */
map(SYS_KILL, do_kill); /* cause a process to be signaled */
map(SYS_GETKSIG, do_getksig); /* signal manager checks for signals */
map(SYS_ENDKSIG, do_endksig); /* signal manager finished signal */
map(SYS_SIGSEND, do_sigsend); /* start POSIX-style signal */
map(SYS_SIGRETURN, do_sigreturn); /* return from POSIX-style signal */

/* Device I/O. */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
Right Ctrl

```

Figura 28 – adicionando na tabela de chamadas

e)

```

Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: syslib.h Modified

/* Forward declaration */
struct reg86u;
struct rs_pci;

#define SYSTASK SYSTEM

/*=====
 * Minix system library.
 *=====*/
PROTOTYPE( int _taskcall, (endpoint_t who, int syscallnr, message *msgptr));
PROTOTYPE( int _kernel_call, (int syscallnr, message *msgptr));
PROTOTYPE( int sys_mykernelcall(unsigned flags, endpoint_t proc_ep));
PROTOTYPE( int sys_abort, (int how, ...));
PROTOTYPE( int sys_enable_iop, (endpoint_t proc_ep));
PROTOTYPE( int sys_exec, (endpoint_t proc_ep, char *ptr,
                        char *aout, vir_bytes initpc));
PROTOTYPE( int sys_fork, (endpoint_t parent, endpoint_t child, endpoint_t *,
                        struct mem_map *ptr, u32_t vm, vir_bytes *));
PROTOTYPE( int sys_newmap, (endpoint_t proc_ep, struct mem_map *ptr));
PROTOTYPE( int sys_clear, (endpoint_t proc_ep));

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
Right Ctrl

```

Figura 29 – adicionando o protótipo na função *sys_mykernelcall.c*

f)

```

Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: com.h Modified

# define SYS_SAFEMAP (KERNEL_CALL + 47) /* sys_safemap() */
# define SYS_SAFEREUMAP (KERNEL_CALL + 48) /* sys_saferevmap() sys_saferev$
# define SYS_SAFEUNMAP (KERNEL_CALL + 49) /* sys_safeunmap() */
# define SYS_GETMCONTEXT (KERNEL_CALL + 50) /* sys_getmcontext() */
# define SYS_SETMCONTEXT (KERNEL_CALL + 51) /* sys_setmcontext() */

# define SYS_UPDATE (KERNEL_CALL + 52) /* sys_update() */
# define SYS_EXIT (KERNEL_CALL + 53) /* sys_exit() */

# define SYS_SCHEDCTL (KERNEL_CALL + 54) /* sys_schedctl() */
# define SYS_STATECTL (KERNEL_CALL + 55) /* sys_statectl() */
# define SYS_MYKERNELCALL (KERNEL_CALL +56) /*sys_mykernelcall()*/
/* Total */
#define NR_SYS_CALLS 56 /* number of kernel calls */

#define SYS_CALL_MASK_SIZE BITMAP_CHUNKS(NR_SYS_CALLS)

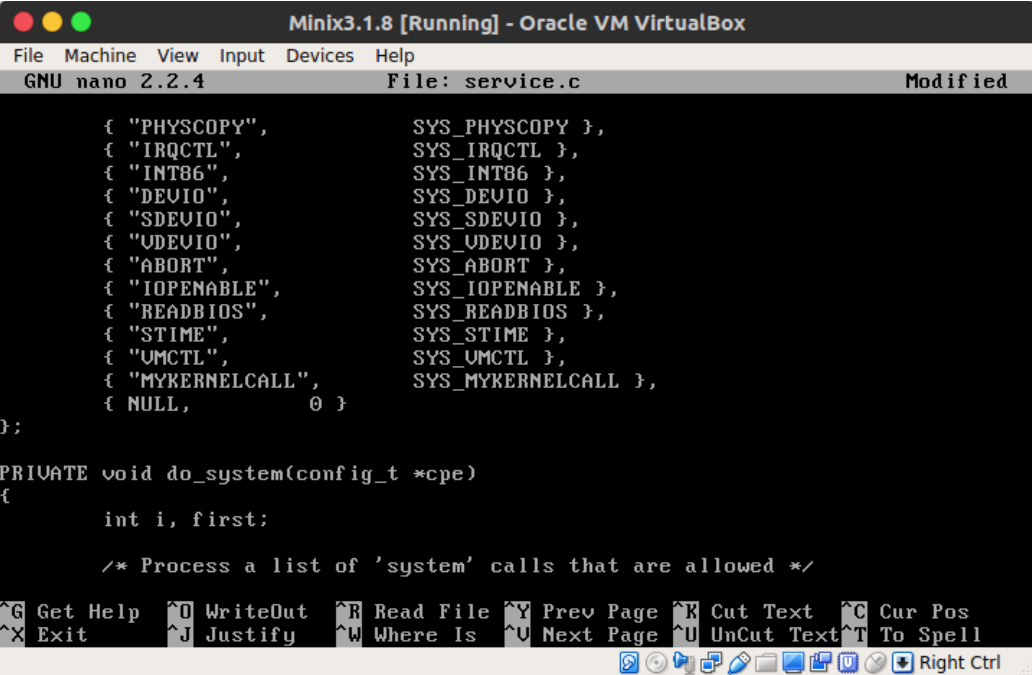
/* Basic kernel calls allowed to every system process. */
#define SYS_BASIC_CALLS \
    SYS_EXIT, SYS_SAFECOPYFROM, SYS_SAFECOPYTO, SYS_USAFECOPY, SYS_GETINFO, \

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
Right Ctrl

```

Figura 30 – adicionando ao vetor de chamadas

g)



```

Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: service.c Modified

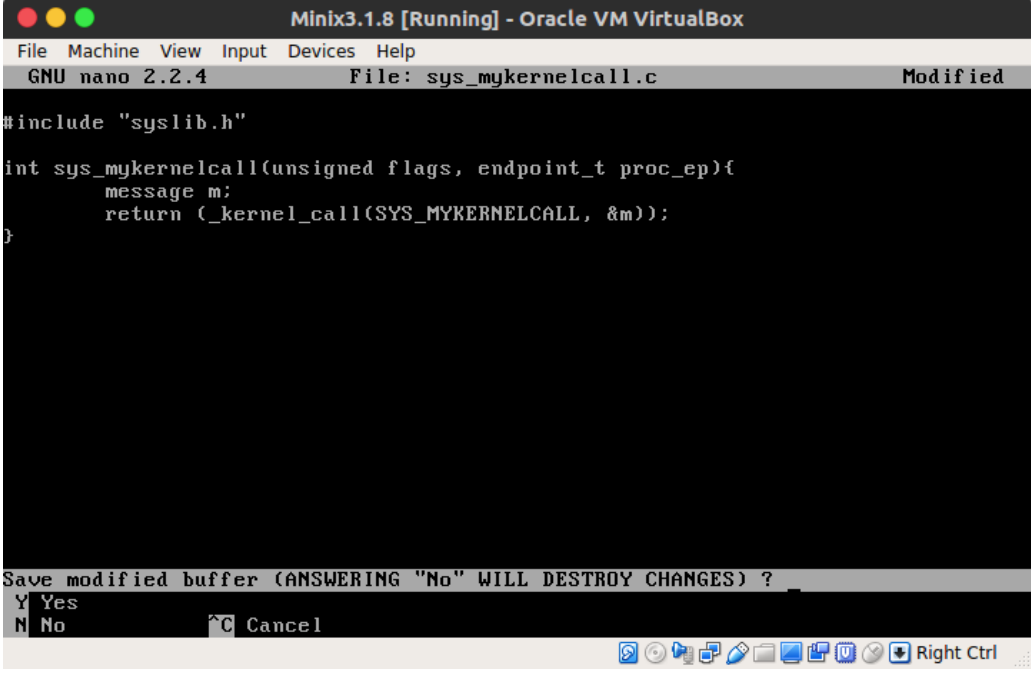
{ "PHYSCOPY",      SYS_PHYSCOPY },
{ "IRQCTL",        SYS_IRQCTL },
{ "INT86",         SYS_INT86 },
{ "DEVIO",         SYS_DEVIO },
{ "SDEVIO",        SYS_SDEVIO },
{ "UDEVIO",        SYS_UDEVIO },
{ "ABORT",         SYS_ABORT },
{ "IOPENABLE",     SYS_IOPENABLE },
{ "READBIOS",      SYS_READBIOS },
{ "STIME",         SYS_STIME },
{ "UMCTL",         SYS_UMCTL },
{ "MYKERNELCALL", SYS_MYKERNELCALL },
{ NULL,           0 }
};

PRIVATE void do_system(config_t *cpe)
{
    int i, first;

    /* Process a list of 'system' calls that are allowed */
}
  
```

Figura 31 – adicionando à tabela de serviços

h)



```

Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: sys_mykernelcall.c Modified

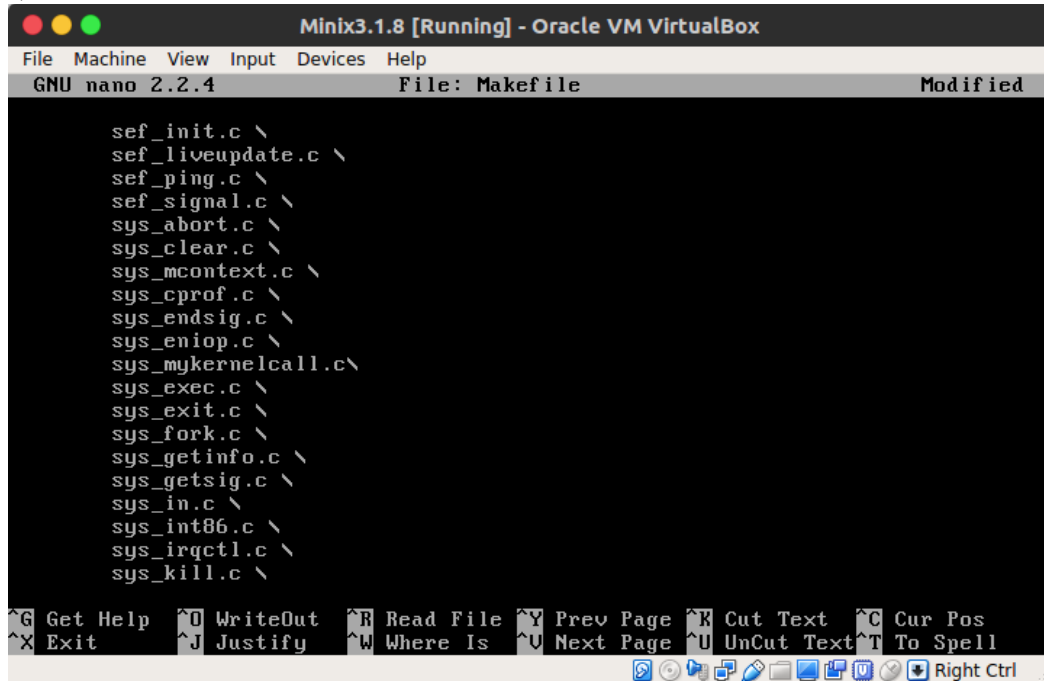
#include "syslib.h"

int sys_mykernelcall(unsigned flags, endpoint_t proc_ep){
    message m;
    return (_kernel_call(SYS_MYKERNELCALL, &m));
}

Save modified buffer (ANSWERING "No" WILL DESTROY CHANGES) ?
Y Yes
N No ^C Cancel
  
```

Figura 32 – implementação da função `sys_mykernelcall.c`

i)



```
Minix3.1.8 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.2.4 File: Makefile Modified

sef_init.c \
sef_liveupdate.c \
sef_ping.c \
sef_signal.c \
sys_abort.c \
sys_clear.c \
sys_mcontext.c \
sys_cprof.c \
sys_endsig.c \
sys_eniop.c \
sys_mykernelcall.c\
sys_exec.c \
sys_exit.c \
sys_fork.c \
sys_getinfo.c \
sys_getsig.c \
sys_in.c \
sys_int86.c \
sys_irqctl.c \
sys_kill.c \

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^U Next Page ^U UnCut Text ^T To Spell
Right Ctrl
```

Figura 33 – adicionando ao *makefile*

j) Implementando os códigos disponibilizados no material, a chamada de *kernel* criada recentemente não funcionou, gerando um erro ao usar o comando *make install*. Portanto, o código não funcionou.

8. Tarefa 8 – Entrada e saída